



# Intern ultrasound arm sketch



## Goal

- Understand code and lab report
- Improve further on reinforcement learning (what aspect ?)
- Write a more descriptive github and also documented report (weekly update progress)
- Maybe even build prototype to see if it works
- Improve other 4 aspects of the main 5



## Current progress 8/6 + 11/6 (notes informal)

- Resources : rewatch recording, github code, image on teams, teams lab report, summary from claude and conversation
- Brain dump :
  - My goal potential : build prototype myself of robot arm
  - Improve reinforcement learning
  - how can i improve it further what is my objective ?
  - documentation regulatory needed ?
  - error rate ?
  - particular area i want to work on ?
  - 5 things choose 1 ?
  - most important part : improvement to RL 2.3 overall algorithm full change robot hardware make it myself and then improve RL
  - <https://github.com/ucheuz/ImprovedUltrasoundRLRobot>
  - 1 pm housden st thomas meet see robot NW lift up to 4 floor
  - Focus on application of project, future work and limitations after last iteration
  - Read a bit more in detail the lab report → aim and application and limitations and details of components and modules used → paste stuff from claude into here (dont understand some parts) → might wanna learn specifically ultrasound, anatomy and reinforcement learning and how code works in claude project actually works (understand what it made)
  - I might wanna learn reinforcement learning, how claude code works and the code behind this works truly so i can write it myself, along with the code for robotics during unibots and for robotics for tutoring, then also learn how to write claude

## SUMMARY 1 – INITIAL GITHUB + LAB REPORT OVERVIEW

PROJECT: ImprovedUltrasoundRLRobot by ucheuz

### FOLDERS IN REPO:

- Collision Avoidance – obstacle detection/avoidance algorithms
- DynamicModelling and EnvironmentRealism – simulation/modeling
- Framework for Integration into Hardware – hardware integration
- One\_Arm\_Robot – single-arm robot implementation
- Two-Arm Robot – dual-arm robot implementation

Language breakdown: ~82% Jupyter Notebook, ~17% C++

### THE ROBOT

A reinforcement learning control system for a dual-arm ultrasound robot designed for autonomous foetal imaging. The physical robot is a V3.4 dual-probe system with 17 motors, 22 joints (kinematics order), controlled by a Qt6 C++ app called USRobotControl. The goal is to train the arms to autonomously position ultrasound probes on a maternal abdomen without collisions.

### PREVIOUS ITERATION ARCHITECTURE

#### Simulation stack:

- PyBullet – physics engine
- Gymnasium – standard RL environment interface
- SAC (Soft Actor-Critic) – off-policy RL algorithm, continuous action spaces + entropy regularisation
- URDF files – define robot physical structure, joint limits, link geometry

## TRAINING PIPELINE – CURRICULUM LEARNING (4 PHASES)

### Phase 1 – Position-only, workspace expansion curriculum (2M steps)

- Target zone expands from small sub-volume to full clinical geometry (15cm depth, 40cm sweep, 22cm vertical)
- Episode length decreases 320 -> 250 steps (forces efficiency)
- Success = closest end-effector within 0.05m
- Terminal reward: +150

### Phase 2 – Tolerance ramping 0.05 -> 0.04m (500k steps)

### Phase 3 – Tolerance ramping 0.04 -> 0.02m (clinical target)

### Phase 4 – Both arms must reach the same target simultaneously

- Reward penalises the "bottleneck" arm (furthest from target)
- Both arms must be within tolerance simultaneously
- Terminal reward: +260

### Reward function (two-arm):

$R_t = R_{\text{progress}} - 0.02 - (0.5 \times N_{\text{collisions}})$   
 $R_{\text{progress}} = (d_{\text{min},t-1} - d_{\text{min},t}) \times 100.0$

## RESULTS FROM PREVIOUS ITERATION

| Metric                 | Baseline (JRun) | Last Iteration          |
|------------------------|-----------------|-------------------------|
| Convergence rate       | 0.96%           | 30% @ 6.5M steps        |
| Collision reduction    | –               | 94% (17 -> 1 vs random) |
| Position error plateau | –               | ~0.09-0.1m              |
| Orientation            | Failed          | Still unresolved        |
| Compute                | 22 hrs/1M steps | Improved (vectorised)   |

## KEY KNOWN LIMITATIONS TO ADDRESS

1. Orientation convergence failed – joint limits (e.g. Joint 6: 1.571-3.578 rad, Joint 8: negative-only) mean randomly sampled orientation targets are physically unreachable.  
Fix: constrain target sampling to reachable workspace.
2. Only position-only solved for two arms – orientation never added to two-arm training.
3. Hardware integration built but not tested – `RLInput` TCP server + IK bypass exists in theory; needs real-robot validation.
4. Still simulation-only – no sim-to-real transfer yet.

## HARDWARE INTEGRATION DESIGN (already built in previous iteration)

The `RLInput` C++ class acts as a TCP server (port 12345). The Python RL model sends JSON `{"joint_angles": [j0...j21]}` messages. The app bypasses the IK solver (RL outputs joint angles directly, not Cartesian poses) and applies joint limit + GJK collision checks before sending to motors.

=====

SUMMARY 2 – FULL SYNTHESIS (GITHUB + LAB REPORT + MEETING TRANSCRIPT + YOUR NOTES)

=====

THE BIG PICTURE (WHY THIS ROBOT EXISTS)

An ultrasound probe has a small field of view. At a certain foetal age, the baby is too big to capture in one image. The solution: a robot holding two probes simultaneously – better field of view + overlapping images = better scan quality. The robot already physically exists in the hospital lab (north wing). It has two arms, 17 motors total, 8 DOF per arm.

THE CONTROL PROBLEM HISTORY:

1. Dr. Housden built the original controller – pure deterministic inverse kinematics (IK). Worked, but had blind spots – configurations it couldn't reach that it should have.
2. Warren spent ~1 year on the first RL attempt. Barely got it working. Neither Warren nor the supervisor were RL experts; learning as they went.
3. The master's group project (ImprovedUltrasoundRLRobot) took it further – they knew RL techniques and improved it significantly. But it still doesn't work perfectly.
4. Now it's your turn.

THE 5 WORKSTREAMS FROM THE GITHUB

| #   | Folder                    | What it is                                        | Status                                      |
|-----|---------------------------|---------------------------------------------------|---------------------------------------------|
| --- | -----                     | -----                                             | -----                                       |
| 1   | Collision Avoidance       | RL env with inter-arm collision detection         | Largely done                                |
| 2   | Dynamic Modelling & Env   | URDF velocity corrections, standalone GUI         | Done (side project)                         |
| 3   | RL Improvements (One-Arm) | SAC + curriculum, position + orientation          | Position works, orientation FAILED          |
| 4   | RL Improvements (Two-Arm) | Cooperative dual-arm, 4-phase curriculum          | Position partial, orientation not attempted |
| 5   | Hardware Integration      | C++ <u>RLInput</u> TCP server, IK bypass, GUI tab | Built, not tested on real robot             |



## WHAT THE PREVIOUS ITERATION ACHIEVED (AND DIDN'T)

### Wins:

- Curriculum learning fixed catastrophic early-training failure (0.96% -> 30% success)
- Collision avoidance works – 17 to 1 collision vs random policy
- Built a framework to eventually drive the real robot
- Fixed Cartesian sampling offset bug from Warren's version
- Improved compute speed (vectorised, progress-based rewards)

### Still broken / unfinished:

- Orientation never solved – joint limits are asymmetric (Joint 6: 1.571-3.578 rad, Joint 8: negative-only), so randomly sampled orientation targets are physically unreachable. Diagnosed but never fixed.
- Two-arm orientation not attempted – relaxed to position-only throughout all phases
- 30% success rate is still too low for clinical use (need near-100%)
- Hardware never tested on the real robot
- No README or documentation – the group used the repo as a data dump

## WHAT YOUR SUPERVISOR TOLD YOU (from meeting transcript)

### Key points:

- "You probably want to take one of them and see if you can make sense of what they've done and then take it forward a little bit further" → Don't try to do all 5. Go deep on one.
- "The most important part of the project for me is the improvements to the reinforcement learning. Section 2.3 and 3.3." → RL is the priority.
- "Collision avoidance – that's possibly done now" → Don't spend time on folder 1.
- "Dynamic modelling and hardware integration are slightly side projects" → Lower priority.
- "They've finished and gone – there's a task of just making sense of exactly what they've left and getting things running" → First job is get the code working locally.
- "I'm not much of a Python expert" → You're the technical lead on the code side.

Physical robot is in the north wing of the hospital lab. Supervisor has a video of it.

Next in-person meeting: Thursday.

## YOUR OWN GOALS (from your notes)

| Priority | Goal                                                 |
|----------|------------------------------------------------------|
| -----    | -----                                                |
| #1       | Understand the code + lab report fully               |
| #1       | Improve the RL algorithm – section 2.3 is the target |
| High     | Write proper documentation + weekly progress reports |
| Medium   | Focus on application, future work, and limitations   |
| Medium   | Pick 1 of the 5 workstreams to go deep on            |
| Stretch  | Build your own hardware prototype                    |
| Stretch  | Improve the other 4 workstreams after the RL one     |

## RECOMMENDED CONCRETE FOCUS

### Immediate (next week or two):

- Get the code running locally – install PyBullet + Gymnasium + Stable Baselines3, run the One-Arm notebook, see training happen
- Read sections 2.3 + 3.3 of the lab report side-by-side with the code

### Core project goal:

- Fix the orientation failure – root cause is known (unreachable targets due to joint limits). This is a concrete, solvable problem that the last group diagnosed but never fixed. Solving it would be a genuine improvement.
- Extend this to two-arm orientation once one-arm works

### Documentation:

- Write proper READMEs + weekly progress logs as you go

### Longer term:

- Once RL works well in simulation, test the hardware integration
- Potentially build your own lighter prototype to verify

## =====

## KEY FILES & PATHS

## =====

Project folder: C:\ultrasoundmark1  
GitHub (prev iter): <https://github.com/ucheu/ImprovedUltrasoundRLRobot.git>  
Lab report + zip: C:\Users\lawch\OneDrive\文件\programming\imports for ultrasound intern\imports\  
Meeting transcript: C:\Users\lawch\OneDrive\文件\programming\imports for ultrasound intern\imports\transcribed\_audio\_to\_transcript.pdf  
Robot location: North wing, hospital lab (supervisor Dr. Housden can show you)  
Hardware: Dual-arm, V3.4, 17 motors, 22 joints, controlled via USRobotControl (Qt6 C++)





## New goal after understanding

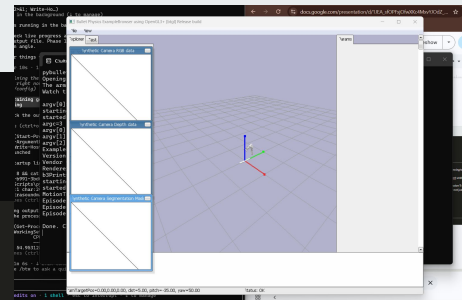
- Improve other 4 aspects of the main 5
- Understand contents of lab report and RL and modules (co-pilot summary and lab report) → git with claude
- Read lab report in more detail to understand aim and limitations
- Run project locally
- Improve on RL algorithm (2.3) → what he meant by orientation failure ?
- AI recommendation : orientation failure and extend to 2 arms and hardware integration (build own prototype)
- Write a readme for github, write improved code for RL, write documented report for weekly update



## Current progress 8/6 + 11/6 (notes informal)

- Currently trying to run it locally → then will improve algorithm
- Currently trying to better understand lab report → will need to paste my notes from AI for technical overview and understanding of lab report and RL here too ...
- AI notes :  
[https://docs.google.com/document/d/1Q1F\\_p1qPE7e2DFuWBPI9L0Kve8i3FMZH\\_Z27goNqhdo/edit?usp=sharing](https://docs.google.com/document/d/1Q1F_p1qPE7e2DFuWBPI9L0Kve8i3FMZH_Z27goNqhdo/edit?usp=sharing) → commented up till 3.3 deep dive (write comments into AI to explain further)
- Claude code running locally : currently downloaded everything either option a or b (need to update conversation and technical progress log before closing every time)
- Claude code for unibots : trying to get arduino to communicate with esp 32 but no success so far

```
Conversation log: C:\ultrasoundmark1\conversation_log_session2.txt
Progress log: C:\ultrasoundmark1\progress_log.txt
Git checkpoint: C:\ultrasoundmark1 (tag: beforeorientationmark2)
```



## Current progress 13/6

- Did meeting, recorded what the robot looks like irl (need to paste it in here asw)
  - Saw the robot, two main questions can it move accurately and efficiently to right position with right rotation (minimum time steps and accurate 80 percent), simulation certain amount of time to get certain amount of results, question two is if can do it better so better algorithm (mainly used IK for first algorithm), can try building it myself, simulation realism so speed faster in simulation compared to real life (idea can unity help with simulation IK ?) real thing is to understand electronics cause do not have blue prints for it (regulatory mainly about proving with experiments if reliable or not) (cool thing rotate internally U scanner) do I want to learn about the force sensors and how ultra scanner works cause he said basically shooting a 2D plane out → then talked about 2 ways of learning either exploring or making it harder and harder ? What is meant by time stamps and reward basically a score and that's what the equation is for to give it a penalty for the score ... main thing get it running locally first then improve RL next Tuesday 12 pm online meeting updates
- Currently main goal trying to run it locally (simulation) → improve RL and orientation (running tests right now)
- Computer running test on how to improve orientation → something to do with euler (i dont understand it)
  - Idea : can use unity 3d and inverse kinematics to do it instead
- Also need to write report by tuesday to update professor housden (maybe get paid discussion ?)
- Goal by tuesday : run it locally simulation, understand solution by AI and mine if works or not just present solution, documentation start
- Paused training at 84 k, still ongoing probably do it when in oxford or monday → read understand explanation and summary update
- Ddpq ???

# Current progress 13/6 (explain problem)

- Sure. Let me break it down from scratch.

---

## The Problem: What is Orientation?

The robot has a probe (the ultrasound head) at the end of its arm. Every time an episode starts, the RL agent gets a target – a specific place in space it needs to move the probe to.

That target has two parts:

- Position – where in 3D space (x, y, z coordinates)
- Orientation – which direction the probe is pointing (is it tilted left? angled down? rotated?)

Phase 1 only trained position – get the probe to the right location, ignore which direction it's pointing. That worked (30% success).

Phase 2 tried to add orientation – get the probe to the right location AND pointing in the right direction. That failed completely.

---

## How Orientation is Described: Euler Angles

To describe "which direction the probe is pointing," you need 3 numbers. One common way is Euler angles – think of it as three separate rotations:

- Roll – spin around the front-to-back axis (like tilting your head side to side)
- Pitch – spin around the left-right axis (like nodding yes)
- Yaw – spin around the up-down axis (like shaking your head no)

Any orientation can be described with these three numbers. The original code picked orientation targets by just grabbing three random numbers within some range – basically spinning the three dials randomly and saying "aim for THIS orientation."

---

## Why That Failed: The Joint Limit Problem

Here's the issue. The robot arm has 10 joints, and each joint has physical limits – it can only rotate within a certain range, just like your elbow can't bend backwards.

Two joints are particularly restricted:

- Joint 6: can only rotate between 90° and 205° – it's *entirely on one side*, it never goes below 90°
- Joint 8: can only rotate between -139° and -40° – it's *entirely negative*, never goes positive

These two joints control a big chunk of how the probe is oriented at the end. Because they're so restricted, the probe can only physically point in a limited subset of all possible directions.

These two joints control a big chunk of how the probe is oriented at the end. Because they're so restricted, the probe can only physically point in a limited subset of all possible directions.

But the original code had no idea about this. It just picked random Euler angles – and most of those random targets pointed in directions that joints 6 and 8 literally cannot achieve. The RL agent was being told "point the probe at this angle" when the arm physically cannot get there. No matter how hard it tried, it could never succeed.

It's like telling someone "point your finger directly behind your own elbow" – it's anatomically impossible, and no amount of training will fix that.

---

## The Fix: Forward Kinematics Sampling

Instead of picking a random orientation and hoping the arm can reach it, my approach works backwards:

1. Pick a random valid joint configuration – randomly set each joint to an angle within its own physical limits. Joint 6 gets something between 90°-205°, joint 8 gets something between -139° and -40°, etc.
2. Ask PyBullet: where does the probe end up? – PyBullet runs the physics and tells us exactly what position and orientation the probe is at, given those joint angles. This is called forward kinematics (FK): given joint angles → compute where the end-effector is.
3. Use that result as the target – the position and orientation we just got from PyBullet become the episode's target.

The key insight: the target is guaranteed reachable because we literally just reached it. We posed the arm in a valid configuration, observed where the probe ended up, and now we're asking the RL agent to reproduce that. There's no guesswork.

---

## Summary Side-by-Side

|                                  | Original (broken)                                             | My fix                                                   |
|----------------------------------|---------------------------------------------------------------|----------------------------------------------------------|
| How target orientation is picked | Random Euler angles from a range                              | Random valid joint config + FK → take the result         |
| Is it reachable?                 | Often no – ignores joint limits                               | Always yes – derived from a real configuration           |
| What the agent experiences       | "Aim here" (impossible) + never succeeds + no learning signal | "Aim here" (definitely reachable) → can succeed + learns |

---

## Summary Side-by-Side

|                                  | Original (broken)                                             | My fix                                                   |
|----------------------------------|---------------------------------------------------------------|----------------------------------------------------------|
| How target orientation is picked | Random Euler angles from a range                              | Random valid joint config + FK → take the result         |
| Is it reachable?                 | Often no – ignores joint limits                               | Always yes – derived from a real configuration           |
| What the agent experiences       | "Aim here" (impossible) + never succeeds + no learning signal | "Aim here" (definitely reachable) → can succeed + learns |

---

That's the full picture. What's your input – does the approach make sense to you, or do you see a problem with it?

• Worked for 30s · 2 shells still running

• recap: Training the ultrasound robot RL arm. Phase 1 position training is running in the background at about 363 steps per minute. Let me know when you want to pause or check progress.

• save this conversatoin and show me path of conversation, then aslo save the technical overview progress thingy

• Let me update the progress log with the training status, write the conversation file, then commit both.

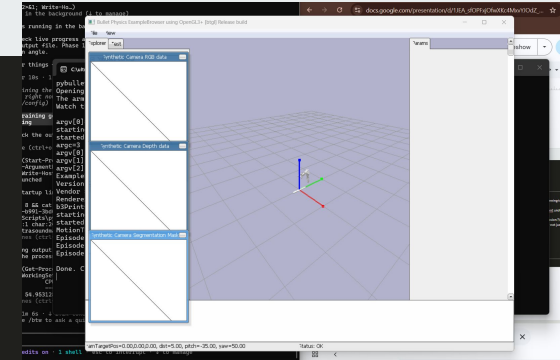


## Current progress 13/6 (still informal notes)

- Building a mini version of hardware in HK (buy components first)
- Who can i talk to if i need components ???
- tue meeting notes : run locally complete, currently running simulation so next task is just to make presentation and progress update look cleaner
  - Need to do : progress update look better
  - Ask : publication and getting paid ?
- Lab report summary and components list on google doc

# 16/6 progress update summary

- Understanding of ultrasound robot big picture complete
- run locally complete on pybullet complete (show through screen visual demo)
- Currently trying 2 methods to improve orientation (euler vs inverse kinematics)
- Next step : improve RL algorithm
- Ask : publication and getting paid ? certain format for updating ?
- Feedback :
  - what we know gap try to understand what they did in the lab report especially details good reward function hard test new methods ... Tuesday 12 pm, algorithm improvement
  - Conference what ? 99 percent or AI efficient to make it impressive voluntary so no pressure ? He don't take it seriously ? Find something where I get to contribute more ?
  - Too scared to ask for payment
  - Love this project, this impact and this money spent can get payment ? What happen if say no ? Project still continue ?







## 23/6 progress (informal)

- Skipped meeting
- running the simulation to teach it how to do orientation correctly and currently paused at step 160k, got one success after 84k steps, with reward stabilizing within 30-60 value range
- Need to continue with phase 2 → understand lab report → make summary slides

- Saved to `C:\ultrasoundmark1\conversation_log_session5.txt`.

Key things noted for next session:

- Phase 1 complete (`run1_model.zip` ✓)
- Phase 2 at 350k/500k steps (70%) - resume from `phase2_ckpt_350000_steps.zip`
- Resume logic to add to `run_phase2.py` is written out in the "Next Session" section



## 29/6 progress (informal)

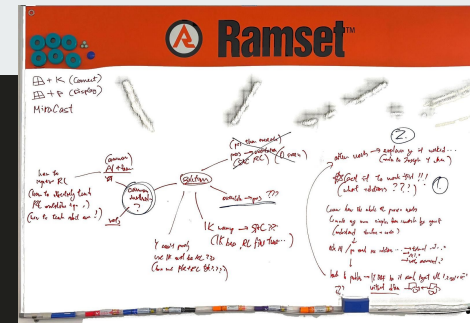
- Brain dump and questions :
  - how entropy introduced into equation --> to explore more than one possible solution to 1 problem, example use of SAC ? --> training robot dogs to walk on rough terrain --> explain a bit better tf a FK is ? IK ? and example use ? show me example of wut URDF ? so wut values am i looking for when RL happening ? how many eps ran with phase 1 and 2 ? --> 19 elements to look at ..., explain a bit better purpose of absolute pos and relative dis ? 45 elements for 2 arms 19 for 1 arm ... (section 9 up till)
  - Reading up till section 9 and 10 right now
  - don't really get section 9 visualise it ? --> reading section 10 rn ..
  - my own braindump to make learning better : AI learning too ineffective the video i watched before ... other flaws of ai to how to increase efficiency
  - then find ways to improve it ? → instead of IK wut abt humans how do we calculate locations and orientation in our heads (muscle memory) high risk of error ?
  - wut abt IK in unity and boston dynamics, i show them how to do it them memorise ? how do they teach their arms to do something usually ?
  - can i build the arm simulator myself ?
  - how did he complete the video trial by himself (housden 2010 demo video of robot arm)
  - ill build the simulator in unity ?



# 30/6 progress (formal)

- Completed phase 1 and phase 2
  - 500 k steps (1700 episodes) for 5 success within 2 cm target
  - Used forward kinematics algorithm to teach it how to reach target (30 cm to within 7 cm) but no success after 500 k steps
  - Mostly used forward kinematics (random angles to see if it works or not) not IK cause too many solutions or none no system or solver like other engineers like unity or boston dynamics to pick best solution
  - Currently learning terminology and continue studying lab report → finding new alternate methods
- Understanding lab report : after project\_summary on doc → project deep dive + professor checklist
- Next step :
  - Previous goals + make it publish worthy and beyond
  - Completing my checklist understanding and reading the deep dive entirely
  - Fully solving orientation problem with new methods
  - Currently : studying inverse kinematics and how it works and unity has its own system to do the calculations ... [https://www.youtube.com/watch?v=SGF3sE2fe\\_k](https://www.youtube.com/watch?v=SGF3sE2fe_k)
- Brain dump and questions : (professor deep dive, checklist, session 6 question confuse, rest before)
  - Idea 30/6 : position + orientation, orientation + position →
  - 7/7 11 pm next week meeting
  - My reflection : reading confusion response by claude to understand better → continue after section 10 for professor deep dive → try to find multiple solutions and try if they work (solution 1 : inverse kinematics, understand it youtube, professor script, unity and boston system try myself) → other alternate methods to try ? like orientation then position or read informal notes and transcript ...
  - Reflection after meeting : IK script in C++ take a look into it, try professor idea, mitigating circumstance defer (re read meeting recording and get transcript, why is it kind of awkward between research meetings between him and professor is it social awareness or something ? who can i ask to improve it better ? he can tell im not genuine and don't really pay effort to project ?) → <https://self-service.kcl.ac.uk/article/KA-01744/en-us> → 3.3 IK real life 2010 (started with warren when know nth about IK then other group that made the RL model)

# 24/7 progress (informal)



- Havent done jack shit → after phase 2 did 100 extra episode analysis of what went wrong and whether brute forcing the testing will make it more accurate over time → recommended to try new method as in adjust formula so inverse kinematics instead of forward kinematics or try orientation than position or something new → and need to add more time onto this (turn this something into research worthy)
- Meeting content : Meeting information intern : Orientation only instead of just position, inverse kinematics → train more, adjust what I try to achieve ... more sophisticated (example position then orientation) more FOCUS !!! Look at the C# script he showed me by adjust position target
- Prompt : last time u ran an extra 100 episodes and i am assuming that what phase 1 and phase 2 and what this did was try to brute force it and run as many episodes as possible to hopefully eventually get the orientation to fit into the 0.02 range, i did some reflection and was thinking of other alternative methods we can try to solve the orientation problem, new method : lets train orientation first then train position to see if there are more success trials, another method : create an inverse kinematics algorithm or something instead of using forward kinematics to get arm to target position with right orientation, can u give me some insight on my thought process, but before that can u visually display what a success trial of the position going to right spot looks like cause i dont really understand what a successful trial is supposed to look like then take a look the script involving the algorithm and see if what im saying is accurate, then lets try to use the methods i mentioned or another new method like adjusting the algorithm entirely to try to achieve our goal of teaching the robot how to do the orientaitona and position to target position and orientation target successfully
- Ideas and thinking process on white board pics phone
- Brain dump : before progressing can claude make a deep dive log to explain what we did in session 8, explain results, explain how this was accomplished (tell professor i am trying to understand why it works), it didnt deform it was just lighting issue, read convo logs + deep dive + ask claude if have any questions

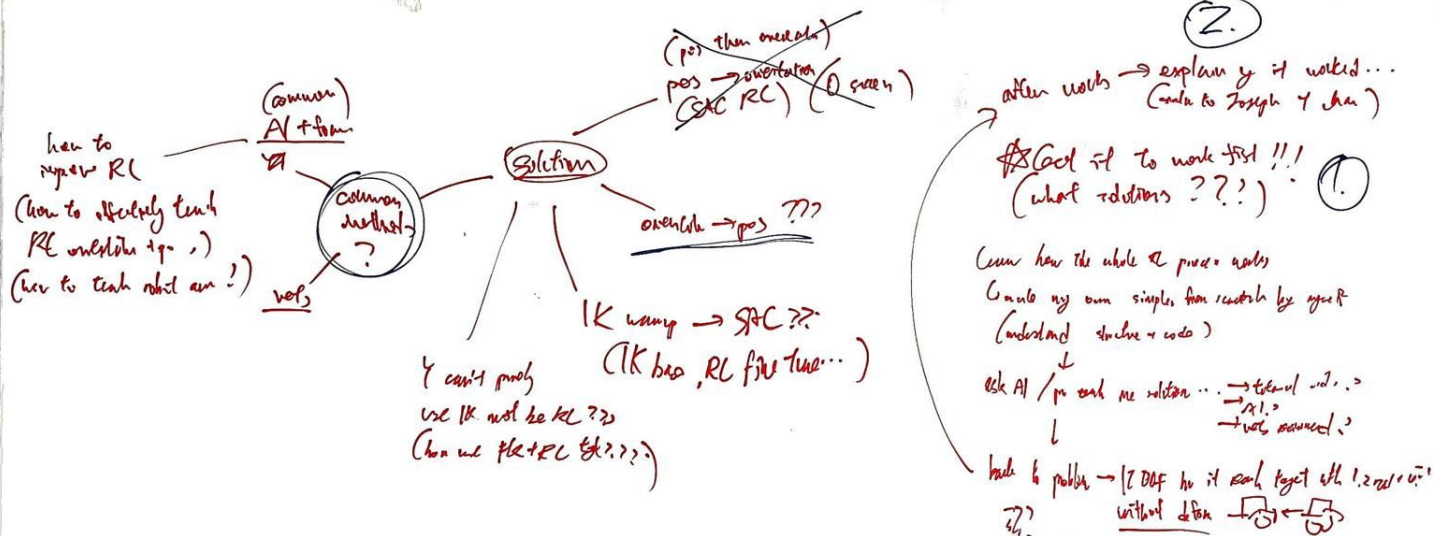


# Ramset™

⊞ + K (Connect)

⊞ + P (Display)

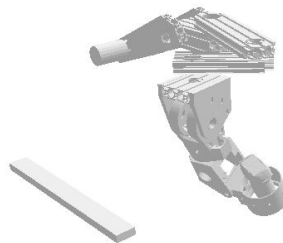
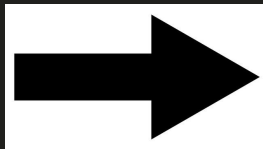
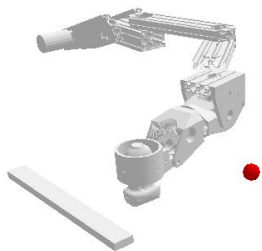
MiraCast





## 28/7 progress (formal)

- Last week : ran 100 episodes to see if the learning curve is there (it is but very slow)
- This week : instead of brute force, tried 2 new methods discussed before
  - Orientation then position after train (isolation then simultaneous, overall sequential)
  - Ik warm start then SAC RL (pretrain orientation and position jointly not staged, then fine tune both variables with SAC)
- Result :
  - Ran 60k time steps isolate only training orientation, 25 % episodes success
  - phase 2 (both orientation and position train) ran 200k time steps, 3 % episodes success (higher than 0 %)
  - Adjusted limits for IK warm set set rail limit to 1 m instead of 0.5 m, wrote IK code, building a 4000 IK dataset → run SAC after ...
- Analysis :
  - Can possibly get even higher success rate if trained orientation RL phase longer as it had an upwards trajectory → the more successful RL model for orientation alone is, the more success episodes could have gotten for the phase that involves both orientation and position (hypothesis)
- Next step and reflection :
  - Currently running IK warm start to see if can achieve similar or even greater success than orientation first position after method
  - Run orientation phase longer to see if more successful episodes
  - Deep dive why methods worked (what question should i be asking ? cause knowledge all over the place)
  - Explain why 2 methods worked and previous didn't + visual demo of what success looks like
  - Explore other alternative methods





## 28/7 progress (informal)

- Orientation not realistic, check orientation target given not too realistic, upside down right side image, careful to train realistic (less about constraints), don't change the rest of the angle constraints, keep training it ... and orientation target given ... (definition of realistic orientation) → earlier time : 5 pm next week !!!
- Next step : on previous slide → paused IK at sth sth steps convo log session 9 ...
- Brain dump : can ask if he thinks it's awkward

Last thing you were doing in Session 9: you'd launched the full IK warm-start run (dataset build → BC pretrain → 200k-step RL fine-tune) and paused it partway through, at step 82,729 of 200,000 (checkpoint saved at step 50,000), so you could let the computer sleep. Results up to that point were noisy/inconclusive - no successes yet, distance oscillating between ~0.04m-0.26m, angle error still mostly 2.1-3.1rad.

Where to pick up next:

1. `run_ik_warmstart.py` doesn't support resuming from checkpoint yet - you'd need a small script loading `ik_warmstart_ckpt_50000_steps.zip` and calling `model.learn(total_timesteps=150000, reset_num_timesteps=False)`, or just relaunch from scratch (either was fine given how inconclusive the trend was).
2. Once trained, you still need to write `evaluate_ik_warmstart.py` (mirroring the existing phase2 evaluator) to get mean dist/angle/success rate, and compare against the old 3% - noting that comparison isn't perfectly apples-to-apples since a joint0 scaling bug fix changed the env after that 3% was recorded.
3. Then, per your original plan: redo orientation-first-then-position with a longer Phase A, compare success rates.
4. Then stop exploring and move to the teaching phase you'd asked for (explain what worked and why) plus a visual/video demo of a full success.
5. Only after that: resume exploring further improvements (e.g. combining IK warm-start with orientation-first).

Two bugs were fixed this session (joint0 action-scaling capped at half range, and an IK label-quality checker comparing against the wrong target) - both already patched in the relevant scripts. Git safety tag `before_the_big_change` exists if you ever need to revert the pre-fix code.



# 16/8 progress (informal)

IK warm-start eval result: 1/100 (1.0%) success rate, mean distance 0.173m, mean angle error 1.787rad.

Comparison against the prior best:

- phase2\_from\_orientation\_first\_model (Session 8): 3% - but trained under the old, buggy /4 joint0 scaling
- IK warm-start (this session): 1% - trained under the corrected /2 scaling, so ~18% of targets that were structurally unreachable before are now reachable, making this technically a harder/fairer test

So on a like-for-like basis IK warm-start alone underperformed the orientation-first curriculum here. Per the original plan, the next step is redoing orientation-first-then-position with a longer Phase A (under the same corrected scaling) so it's a fair fight - that'll tell us whether orientation-first still wins post-fix, or whether both methods land in the same low single digits now that the fix removed some easy wins. Want me to kick that off?

1. Redo orientation-first-then-position with a longer Phase A - run\_orientation\_first\_phase.py already has the joint0 fix applied and is ready to go. Compare its success rate against this session's 1% (both now under the corrected scaling, so it'll be a fair comparison for the first time).
2. STOP exploring new methods at that point. Switch to the teaching phase you explicitly asked for - explain what's been done and why it worked, in plain terms, since you said your own understanding is "all over the place."
3. Visual demo (image, ideally video) of a full successful trial - position and orientation both landing. Was also going to consider switching visualize\_success.py's renderer to ER\_BULLET\_HARDWARE\_OPENGL at that point, since the current renderer makes the T-slot extrusion links look striped/stretched.
4. Only after that: resume exploring further improvement ideas - e.g. combining IK warm-start with the orientation-first curriculum, which was explicitly deferred to this later stage.

So the immediate next step is #1 - the longer orientation-first Phase A redo.

New best result: 6/100 (6.0%) success - mean dist 0.1665m, min dist 0.0865m, mean angle 1.536rad.

This is the number that matters most so far, because it's the first genuinely fair, apples-to-apples comparison: both this and IK warm-start ran under the corrected joint-scaling fix.

| Method                                         | Scaling    | Success |
|------------------------------------------------|------------|---------|
| Orientation-first (original, 60k Phase A)      | old, buggy | 3%      |
| IK warm-start                                  | corrected  | 1%      |
| Orientation-first (longer, 200k Phase A) - new | corrected  | 6%      |

So on equal footing, orientation-first clearly wins over IK warm-start (6x), and the longer Phase A also beat the original 3% despite running on the harder corrected task. Worth updating both published Artifacts with this result, and possibly re-recording the visual demo against this stronger model since it now has double the old model's success rate. Want me to do either of those, or move to something else?



## 16/8 (content for understanding needed)

- Objective for project
- Project set up
- How to build a RL model (concepts needed) : simulator, gym environment, reward design, training loop
- Building RL model (technical)
- Problem and suggested solution : extended training (concepts)
- Building solution (technical)
- Problem and suggested solutions : orientation first (concept)
- Building solution (technical)
- Suggested solutions : IK warm up set (concepts)
- Building solution (technical)
- Suggested solutions : extended orientation first (concept)
- Building solution (technical)
- Entire timeline and summary of current results
- Bugs found during debugging
- Next steps (concept and brief technical)
- Timeline : [https://claude.ai/code/artifact/cb5c8d61-ad8f-40d2-8fbd-c09e6d991d23?via=auto\\_preview](https://claude.ai/code/artifact/cb5c8d61-ad8f-40d2-8fbd-c09e6d991d23?via=auto_preview)
- Set up RL : [https://claude.ai/code/artifact/fe57acba-8f2d-467c-a6d4-ef3e150d1378?via=auto\\_preview](https://claude.ai/code/artifact/fe57acba-8f2d-467c-a6d4-ef3e150d1378?via=auto_preview)



## 16/8 (next step : documentation)

- What i did and Results
  - IK warm up : 1 percent total success rate, mean distance of 0.173 m, mean angle error of 1.787 rad
  - Orientation first phase extension : stuck at 20.9 % compared to previous 19.6 % after 200,000 steps → final success rate : 6 % (highest so far)
- Evaluation and interpretation
  - Broke 0 percent bottleneck, but so far has not been able to reach double digital or 15-20 % deterministic success / landing within 3 cm range reliably (80-90% time)
  - Reaching higher success rate seems improbable as so far has been training 200,000-500,000 steps per run while priori students got a success rate of 30 % after 6.5 million steps
  - Explain my solutions in detail first then continue finding alternate solutions after
- Next steps
  - Priority 1 : complete detailed lab report and documentation (get some advice on how to do so ?, show GD)
  - Alternate solution : combine IK set up with orientation first curriculum
  - Have not accounted for : realistic reach angles ?, awkward ? (expected workspace of robot, direction it will be facing, point downwards into the person, visual demo show underneath (pretend there's a person there)
  - Coordinate system of probe (world space) : z=up, y=up in image (want y to be down in world space)
  - Come up with more solutions to reach higher success rate ? or is it high enough ?